



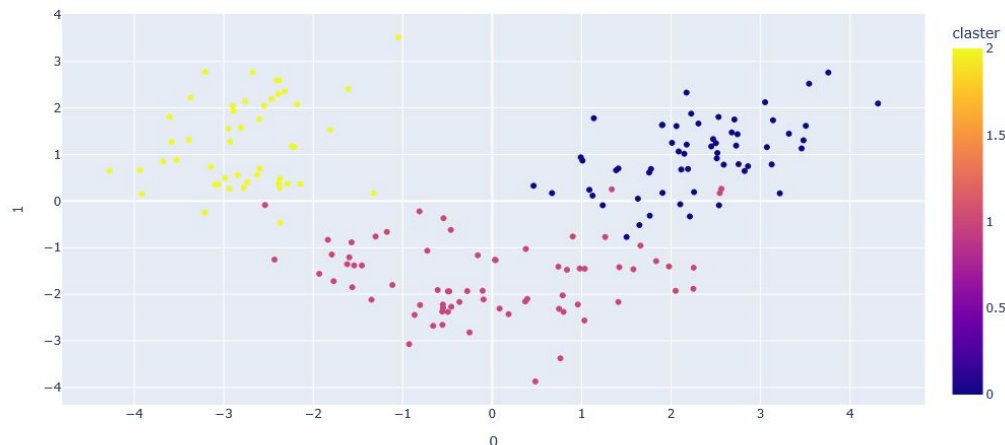
ІІТМО

Лабораторные работы 1-2
Команда 14

Постановка задачи: классификация датасет Wine (13 признаков, 3 класса)



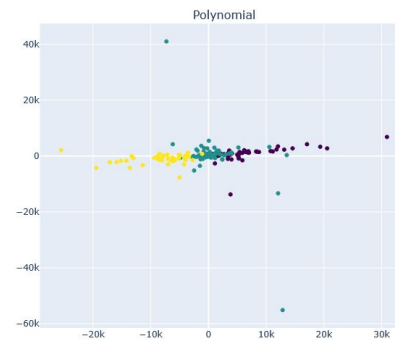
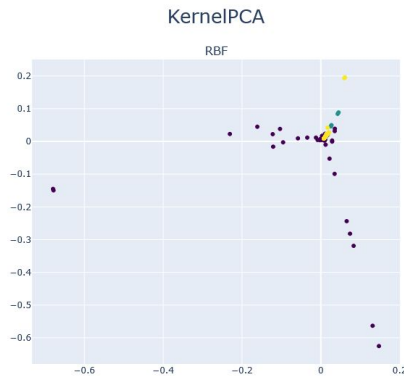
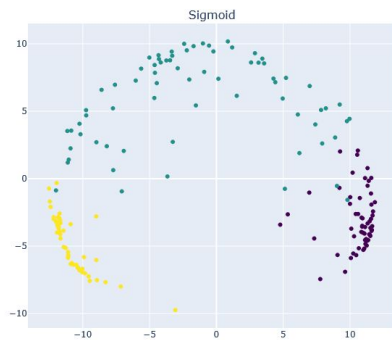
Проекция на первые 2 компоненты Wine



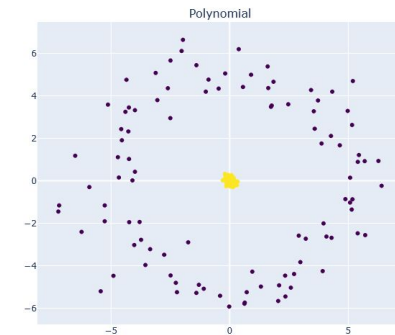
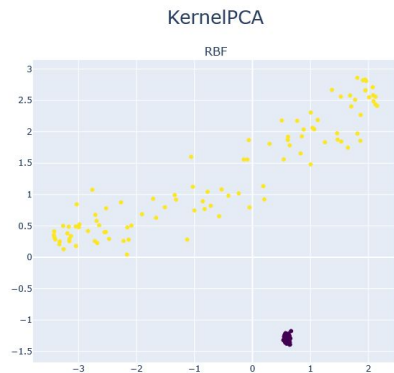
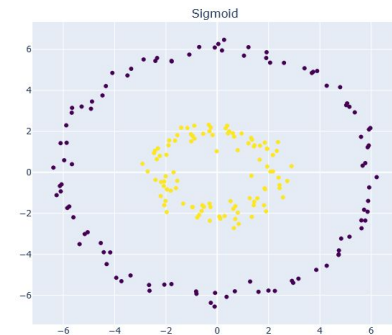
- Wine
basic f1 = 0.97
5 pca f1 = 0.92

KernelPCA

Лабораторная 1



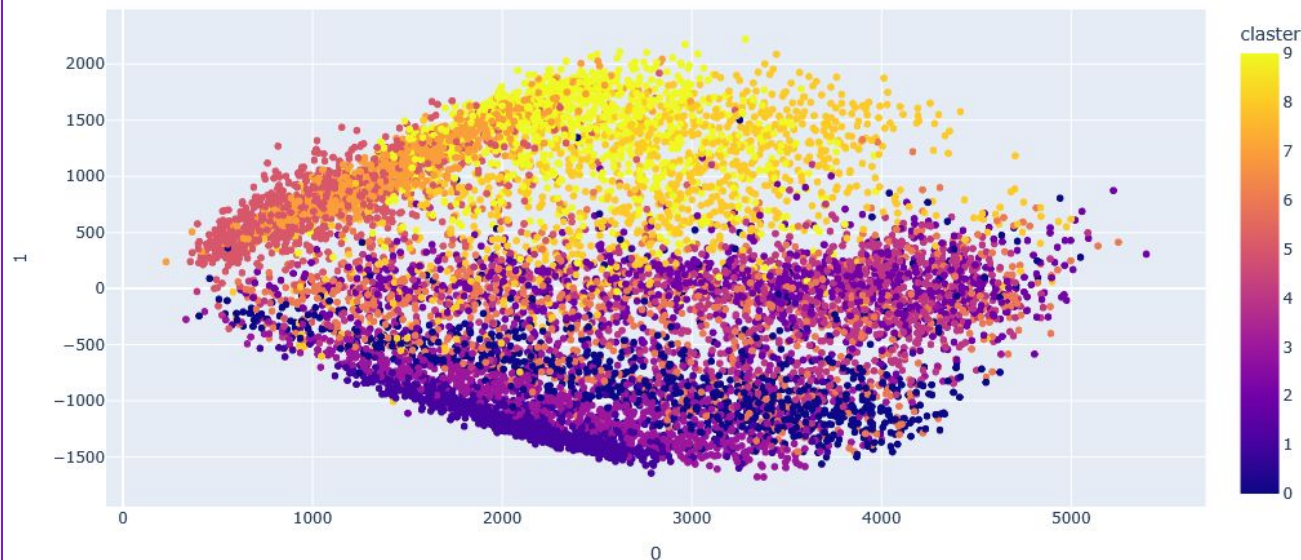
- Wine  
basic f1 = 0.97
sigmoid f1 = 0.95



- Make circles
basic f1 = 0.5
rbf f1 = 1

Сжатие MNIST (784 признака 10к записей)

Проекция MNIST на первые 2 компоненты



basic f1 = 0.71
150 PCA f1 = 0.66

Лабораторная 1



Реализация альтернативных методов сингулярного разложения



Л4 размерность 75000 * 8

	method	n_components	pca_time	lr_time	total_time	accuracy	explained_variance
0	classic	6	0.135	0.167	0.302	0.9332	0.8413
1	randomized	5	0.802	0.145	0.947	0.9331	0.8211
2	iterative	6	1.257	0.297	1.554	0.9332	0.8359
3	jacobi	7	1.930	0.312	2.242	0.9332	0.8750

MNIST сэмпл 300. кол-во компонент фикс - 20 (модель RFC)
сжатие 196 -> 20 (9.8 раз)

	method	pca_time	rf_time	total_time	accuracy	f1_score	explained_variance
0	classic	26.052	0.282	26.334	0.7111	0.7015	0.7790
1	randomized	25.600	0.294	25.894	0.6444	0.6185	0.8430
2	iterative	0.170	0.281	0.451	0.6889	0.6800	0.7788
3	block	33.285	0.406	33.691	0.6667	0.6592	0.7790
4	jacobi	42.854	0.266	43.120	0.6889	0.6794	0.6416

MNIST сэмпл 300. авто выбор компонент по порогу 80%. (модель RFC)

	method	n_comp	comp	pca_t	rf_t	total	acc	f1	var
0	classic	22	8.9	24.815	0.282	25.097	0.622	0.604	0.804
1	randomized	17	11.5	26.195	0.271	26.466	0.667	0.662	0.807
2	iterative	22	8.9	0.171	0.285	0.456	0.711	0.703	0.804
3	block	22	8.9	25.662	0.395	26.057	0.722	0.722	0.804
4	jacobi	42	4.7	23.692	0.306	23.998	0.656	0.647	0.801

Лабораторная 1

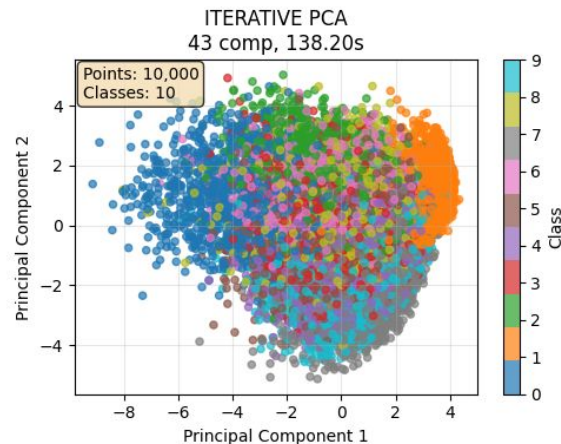
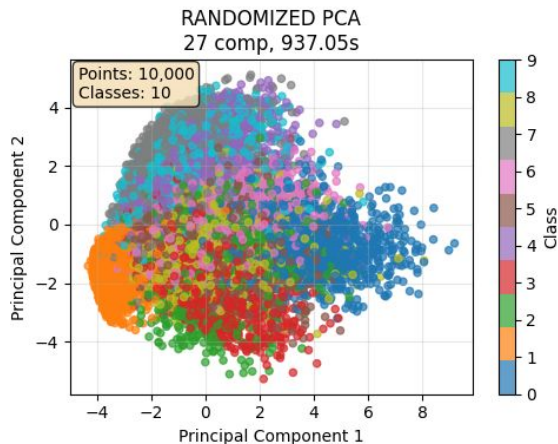
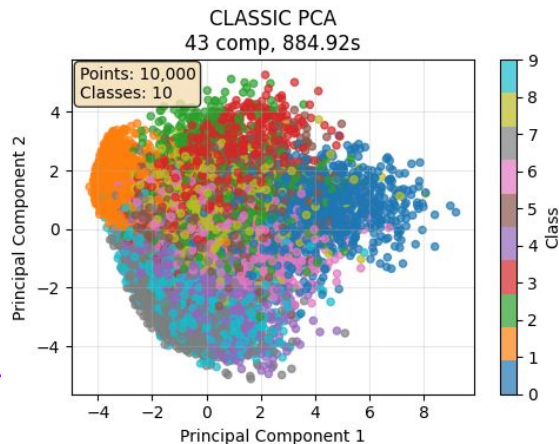


Реализация альтернативных методов сингулярного разложения

MNIST выборка 10000 * 784

	method	n_components	compression	pca_time	rf_time	total_time	accuracy	f1_score	explained_variance
0	classic	43	18.2	757.761107	3.935104	761.696210	0.899667	0.899601	0.801539
1	randomized	27	29.0	1042.130871	3.457404	1045.588275	0.896667	0.896651	0.806181
2	iterative	43	18.2	148.182694	5.664408	153.847103	0.901667	0.901593	0.800599

Время обучения
модели на
оригинальных
данных: 6.239 сек
Accuracy: 0.9307
F1-score: 0.9306



Постановка задачи: минимизация целевой функции



$$f(x) = \frac{1}{2}[(x_1)^2 + (x_1 - x_2)^2 + (x_2 - x_3)^2 + x_3^2] - x_1$$

Для применения методов оптимизации нужно показать дважды дифференцируемость и выпуклость целевой функции. Выпишем частные производные:

$$\frac{df}{dx_1} = \frac{1}{2}[2x_1 + 2(x_1 - x_2)] - 1 = 2x_1 - x_2 - 1$$

$$\frac{df}{dx_2} = -(x_1 - x_2) + (x_2 - x_3) = -x_1 + 2x_2 - x_3$$

$$\frac{df}{dx_3} = -(x_2 - x_3) + x_3 = -x_2 + 2x_3$$

$$\frac{d^2 f}{dx_1 dx_2} = -1$$

$$\frac{d^2 f}{dx_1 dx_3} = 0$$

$$\frac{d^2 f}{dx_2 dx_3} = -1$$

$$\frac{d^2 f}{dx_1^2} = 2$$

$$\frac{d^2 f}{dx_2^2} = 2$$

$$\frac{d^2 f}{dx_3^2} = 2$$

Лабораторная 2



BFGS

```
Iteration 1:, f(x) = 9697.9375, ||grad_f(x)|| = 112.22883987638828
Iteration 2:, f(x) = 966.950538276737, ||grad_f(x)|| = 59.178158627820046
Iteration 3:, f(x) = 14.799133015709911, ||grad_f(x)|| = 10.034380986344912
Iteration 4:, f(x) = -0.364037694432988, ||grad_f(x)|| = 0.26303398007623185
Iteration 5:, f(x) = -0.37499999988299226, ||grad_f(x)|| = 1.8962530153903324e-05
Iteration 6:, f(x) = -0.3749999999999999, ||grad_f(x)|| = 2.1859361129954452e-08
[0.75000001 0.50000001 0.25      ]
```

Ответ x: [0.75000001 0.50000001 0.25]
Значение функции в точке x: -0.3749999999999999

L-BFGS

```
Iteration 1, f(x) = 9697.9375, ||grad_f(x)|| = 112.22883987638828
Iteration 2, f(x) = 3660.8058603482004, ||grad_f(x)|| = 79.33713262466172
Iteration 3, f(x) = 358.28285536915104, ||grad_f(x)|| = 36.25669934900724
Iteration 4, f(x) = 7.551300605151125, ||grad_f(x)|| = 6.057727256872553
Iteration 5, f(x) = -0.29643529183353134, ||grad_f(x)|| = 0.359549420958124
Iteration 6, f(x) = -0.374392083714639, ||grad_f(x)|| = 0.047201242774483274
Iteration 7, f(x) = -0.3749999025114758, ||grad_f(x)|| = 0.0006564760073752702
Iteration 8, f(x) = -0.37499999999144035, ||grad_f(x)|| = 5.948189756116432e-06
Iteration 9, f(x) = -0.3749999999999756, ||grad_f(x)|| = 9.916695901688007e-08
[0.75000003 0.50000006 0.25000001]
```

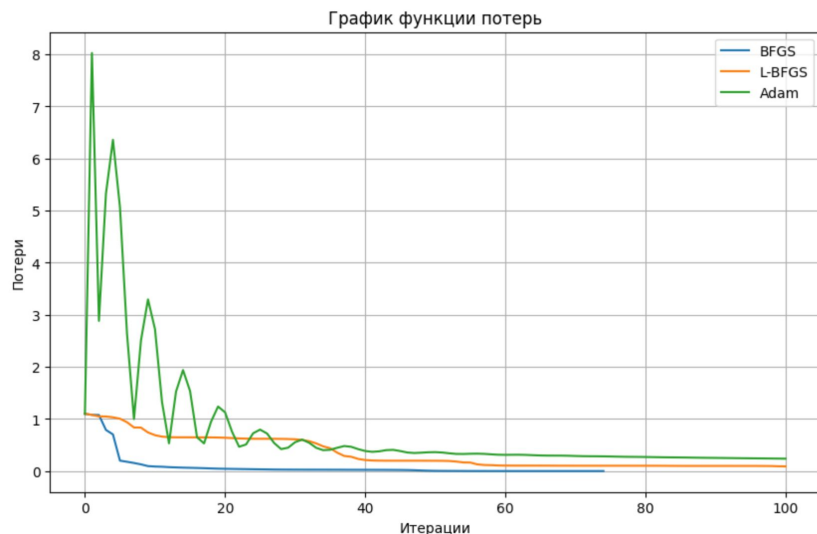
Ответ x: [0.75000003 0.50000006 0.25000001]
Значение функции в точке x: -0.3749999999999756



Лабораторная 2



Сравнение функций оптимизации



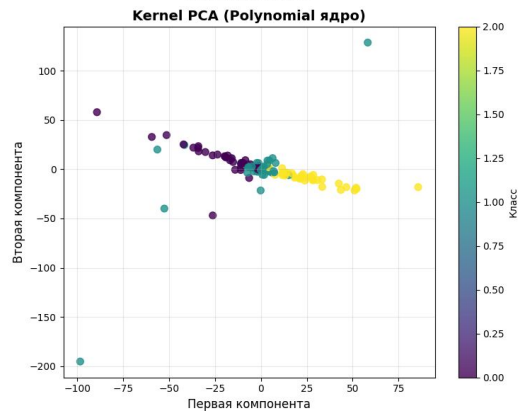
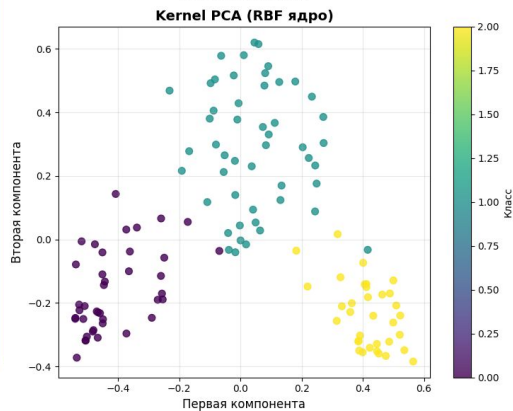
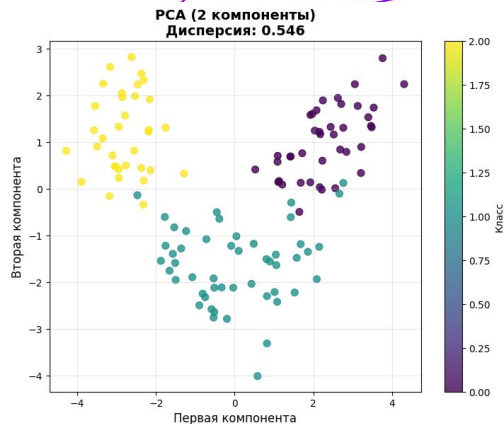
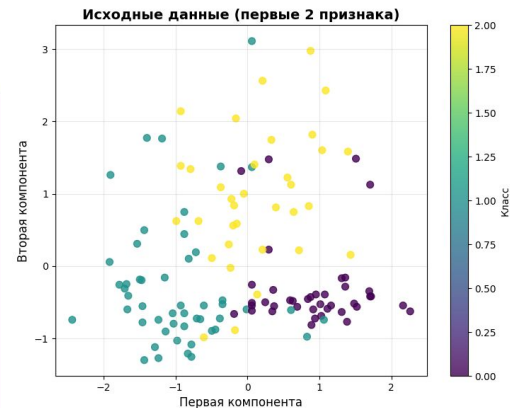
min_iter = 100 epsilon = 1e-6

Оптимизатор	Accuracy
BFGS	1.00
L-BFGS	0.97
Adam	0.97
Logistic Regression	0.97

Спасибо
за внимание!

it's **MO** *re than a*
UNIVERSITY

Приложение



=== PCA с 2 компонентами ===

Объясненная д: 0.5460

Доля по компонентам: [0.36534816 0.18061255]

Точность после PCA: 0.9630

	precision	recall	f1-score	support
0	1.00	0.89	0.94	19
1	0.91	1.00	0.95	21
2	1.00	1.00	1.00	14
accuracy			0.96	54
macro avg	0.97	0.96	0.97	54
weighted avg	0.97	0.96	0.96	54

=== PCA с 3 компонентами ===

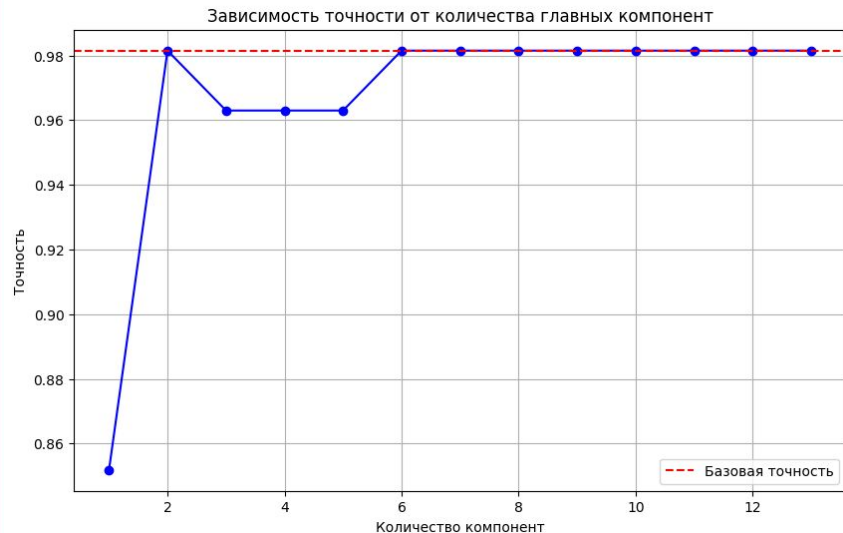
Объясненная д: 0.6628

Доля по компонентам: [0.36534816 0.18061255 0.11686462]

Точность после PCA: 0.9630

	precision	recall	f1-score	support
0	1.00	0.89	0.94	19
1	0.91	1.00	0.95	21
2	1.00	1.00	1.00	14
accuracy			0.96	54
macro avg	0.97	0.96	0.97	54
weighted avg	0.97	0.96	0.96	54

Приложение



=== PCA с 5 компонентами ===

Объясненная д: 0.8135

Доля по компонентам: [0.36534816 0.18061255 0.11686462 0.07930205 0.07138901]

Точность после PCA: 0.9630

	precision	recall	f1-score	support
0	0.95	1.00	0.97	19
1	1.00	0.95	0.98	21
2	0.93	0.93	0.93	14
accuracy			0.96	54
macro avg	0.96	0.96	0.96	54
weighted avg	0.96	0.96	0.96	54

=== PCA с 10 компонентами ===

Объясненная д: 0.9622

Доля по компонентам: [0.36534816 0.18061255 0.11686462 0.07930205 0.07138901 0.04610997 0.03442149 0.02730303 0.02105357 0.01980785]

Точность после PCA: 0.9815

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	0.95	0.98	21
2	0.93	1.00	0.97	14
accuracy			0.98	54
macro avg	0.98	0.98	0.98	54
weighted avg	0.98	0.98	0.98	54

Обучение через оптимизаторы

```
▶ y_pred = np.argmax(softmax(X @ W_opt_bfgs), axis=1)  
print(classification_report(y, y_pred))
```

...		precision	recall	f1-score	support
	0	1.00	1.00	1.00	59
	1	1.00	1.00	1.00	71
	2	1.00	1.00	1.00	48
	accuracy			1.00	178
	macro avg	1.00	1.00	1.00	178
	weighted avg	1.00	1.00	1.00	178

```
y_pred = np.argmax(softmax(X @ W_opt_lbfgs), axis=1)  
print(classification_report(y, y_pred))
```

		precision	recall	f1-score	support
	0	0.93	0.93	0.93	59
	1	0.93	0.92	0.92	71
	2	0.96	0.98	0.97	48
	accuracy			0.94	178
	macro avg	0.94	0.94	0.94	178
	weighted avg	0.94	0.94	0.94	178

```
▶ y_pred = np.argmax(softmax(X @ W_opt_adam), axis=1)  
print(classification_report(y, y_pred))
```

...		precision	recall	f1-score	support
	0	0.93	0.93	0.93	59
	1	0.93	0.92	0.92	71
	2	0.96	0.98	0.97	48
	accuracy			0.94	178
	macro avg	0.94	0.94	0.94	178
	weighted avg	0.94	0.94	0.94	178

```
etalon = LogisticRegression(max_iter=100)  
etalon.fit(X, y)  
print(classification_report(y, etalon.predict(X)))
```

		precision	recall	f1-score	support
	0	1.00	0.95	0.97	59
	1	0.95	0.97	0.96	71
	2	0.96	0.98	0.97	48
	accuracy			0.97	178
	macro avg	0.97	0.97	0.97	178
	weighted avg	0.97	0.97	0.97	178